

Recap

Suppose a large-scale, fault-tolerant quantum computer becomes available. Which of the following statements about its impact on today's cryptography is correct?

- A. Shor's algorithm breaks RSA and elliptic-curve cryptography, while AES-256 remains practically secure against Grover's algorithm.
- B. Grover's algorithm breaks RSA and elliptic-curve cryptography, while Shor's algorithm halves the effective key length of AES.
- C. Both RSA and AES-256 are completely broken, so all currently deployed cryptography must be replaced.
- D. Quantum computers threaten only symmetric ciphers; public-key schemes based on factoring are unaffected.

Announcement

- HW1 has been released on Blackboard. Due Sep 16, 2026, 11:59 PM UMBC (next Wednesday)

if any homework questions felt tough, no worries. We'll cover those concepts and similar ones over these two lectures

- If you have any questions or need help, you are welcome to stop by office hours

Announcement

- **Today we have Quiz 1.** The questions are on the sheet I just handed out
- **Good news:** if you follow along with me in class and work through the problems step by step, **I will reveal the answers** at the end of the lecture
- In other words, as long as you attend, work through the quiz with me, and turn in your sheet, you will receive **full credit** for this quiz. It's worth 2 points of your final course grade

Announcement

- We **will not** have in-person lectures on Monday, October 26 and Wednesday, October 28, 2026
 - I will be attending the ACM MobiCom conference that week to present an accepted paper
- For the above two lectures, I'd like to hear which online format you prefer. Feel free to write your choice on the back of today's quiz sheet:
 - (1) Live class on Webex, at our regular class time
 - (2) Pre-recorded video lectures posted on Blackboard, which you watch on your own schedule
 - (3) No Webex, no videos. Use those two class sessions to work with teammates on the course project

Announcement

Optional: feel free to also jot down any thoughts or suggestions about this course so far.

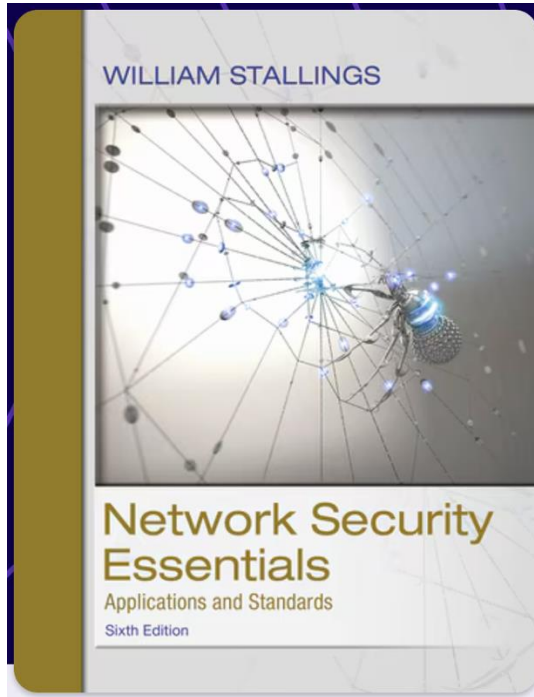
For example:

- topics you'd like us to cover
- whether you'd prefer more concepts or more hands-on experiments
- whether the pace feels a little bit fast or slow
- ...

Symmetric Encryption

Dr. Yiwen Hu

Acknowledgement



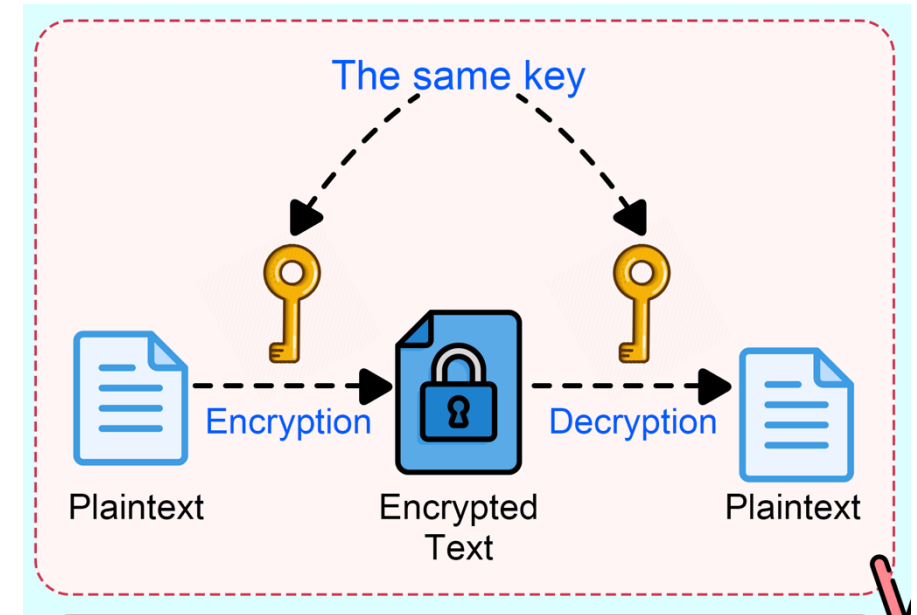
You can find it on Blackboard

The content of slides is from the textbook and Dr. Hu's supplementary materials

Reminder: Please do not post course materials online or share them with others without the instructor's permission

Overview

- Symmetric Cryptography
 - Five types of attacks on encrypted messages
- **Feistel network**
 - Many symmetric algorithms have similar design
- **Block ciphers**
 - DES, **AES**
- **Stream cipher**
 - **RC4**



Types of Attacks on Encrypted Messages

Type of Attack	Known to Cryptanalyst
Ciphertext only	•Encryption algorithm •Ciphertext to be decoded
Known plaintext	•Encryption algorithm •Ciphertext to be decoded •One or more plaintext-ciphertext pairs formed with the secret key
Chosen plaintext	•Encryption algorithm •Ciphertext to be decoded •Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key
Chosen ciphertext	•Encryption algorithm •Ciphertext to be decoded •Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key
Chosen text	•Encryption algorithm •Ciphertext to be decoded •Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key •Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key

Hard



Easy

Question

- An attacker intercepts an encrypted file sent via email. The attacker does not have the key, but they know the file is a standard Microsoft Word document (.docx). Because all .docx files start with the same underlying file signature ("PK"), the attacker uses this known signature to help determine the encryption key.

Which type of cryptanalytic attack is this?

- A) Brute-Force Attack
- B) Ciphertext-Only Attack
- C) Known Plaintext Attack

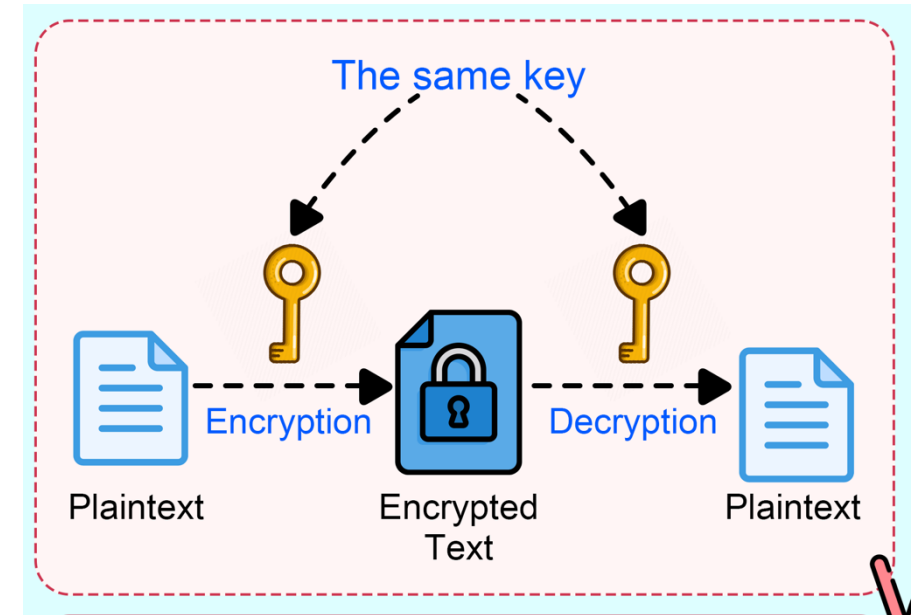
Type of Attack	Known to Cryptanalyst
Ciphertext only	•Encryption algorithm •Ciphertext to be decoded
Known plaintext	•Encryption algorithm •Ciphertext to be decoded •One or more plaintext-ciphertext pairs formed with the secret key
Chosen plaintext	•Encryption algorithm •Ciphertext to be decoded •Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key
Chosen ciphertext	•Encryption algorithm •Ciphertext to be decoded •Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key
Chosen text	•Encryption algorithm •Ciphertext to be decoded •Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key •Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key

Computationally Secure Encryption Schemes

- Encryption is computationally secure if:
 - Cost of breaking cipher exceeds the value of information
 - Time required to break cipher exceeds the useful lifetime of the information
- Very difficult to estimate the amount of effort required to break
 - Thus, we usually estimate time/cost of a brute-force attack

Overview

- Symmetric Cryptography
 - Five types of attacks on encrypted messages
- **Feistel network**
 - Many symmetric algorithms have similar design
- **Block ciphers**
 - DES, 3DES, **AES**
- **Stream cipher**
 - **RC4**



Classical Feistel Network

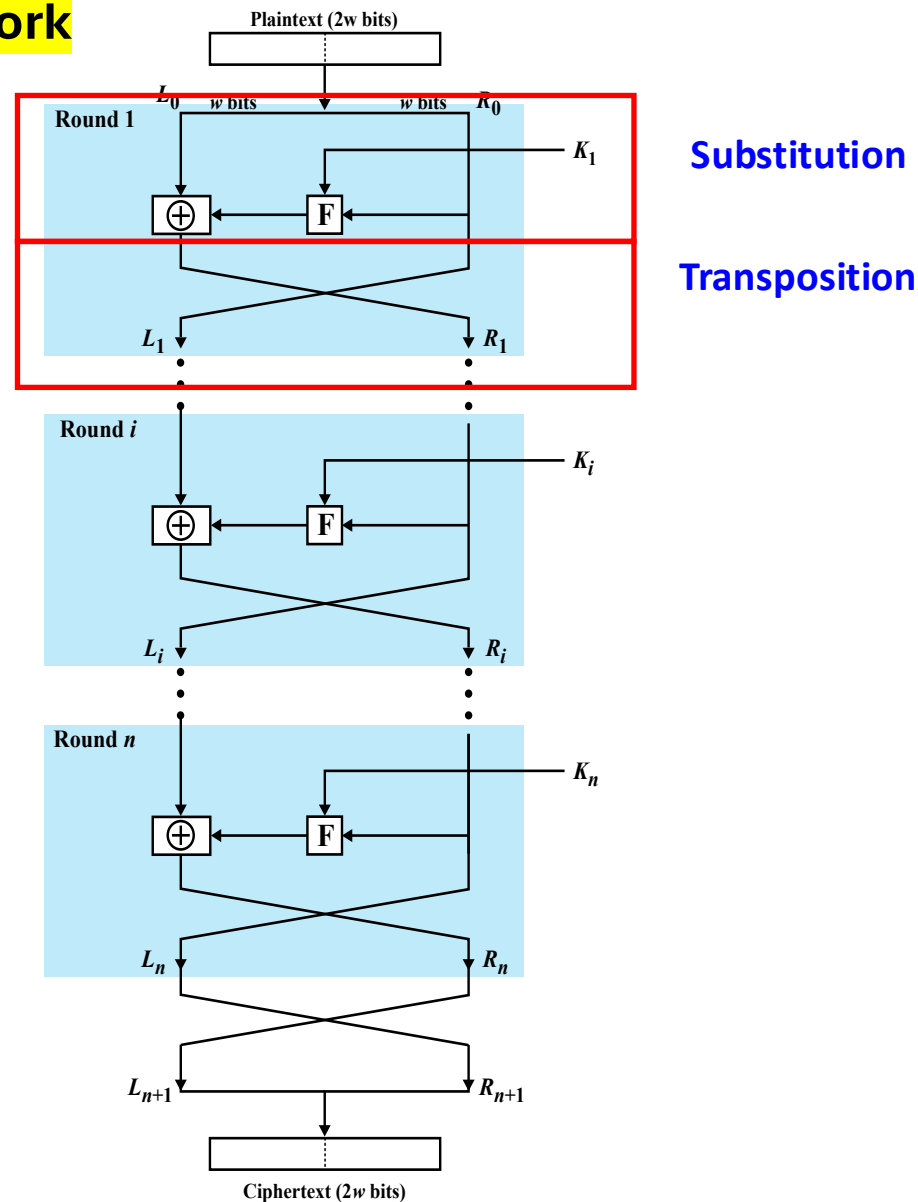


Figure 20.1 Classical Feistel Network

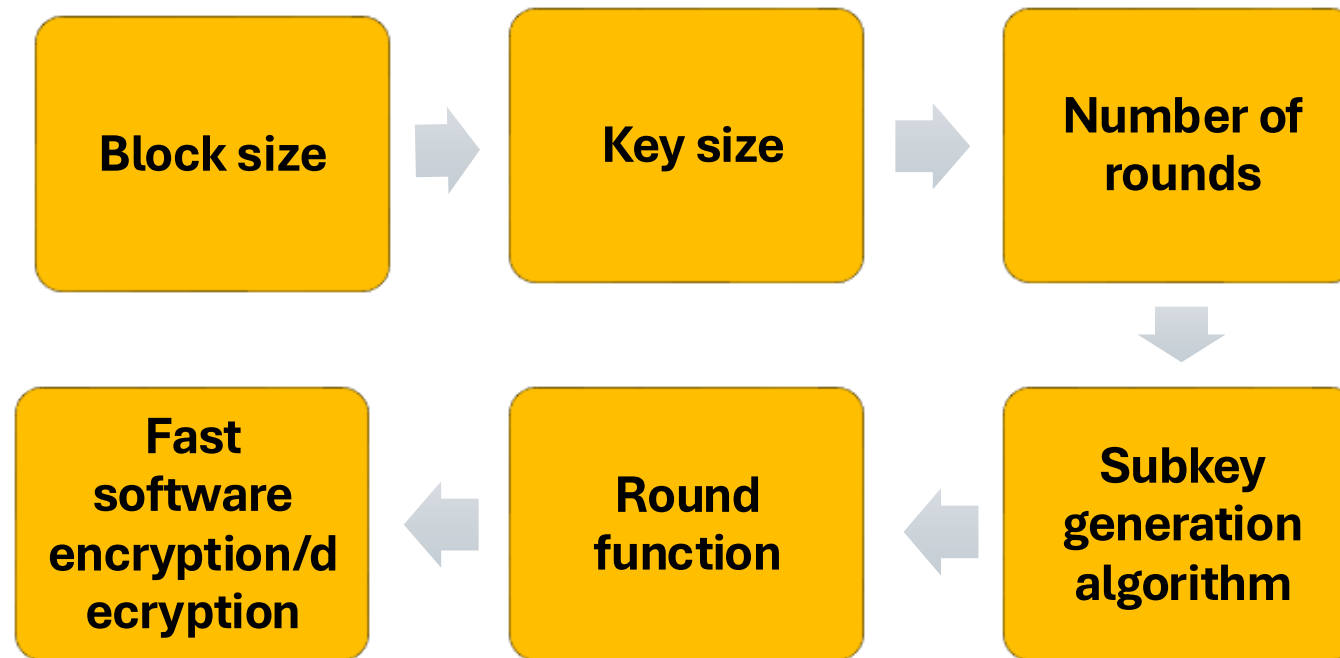
- Consists of **multiple rounds**
 - Using the same round function F
 - In each round, a different key is used to configure the round function F
- In each round, two operations are made
 - **Substitution**
 - **Only change left w-bit data**
 - Sending right w-bit of data to the round function F
 - Taking XOR of the output of F (right w-bit data) and left w-bit data
 - **Transposition**
 - **Swap** the left w-bit data and right w-bit data **for next round**
- $R_0 \Rightarrow L_1$
- **$L_0 \text{ XOR } F(R_0) \Rightarrow R_1$**



Block Cipher Algorithms

Block Cipher Structure

- Symmetric block cipher consists of:
 - A sequence of rounds
 - With **substitutions** and **transpositions** controlled by a **key (and its subkeys)**
- Parameters and design features:



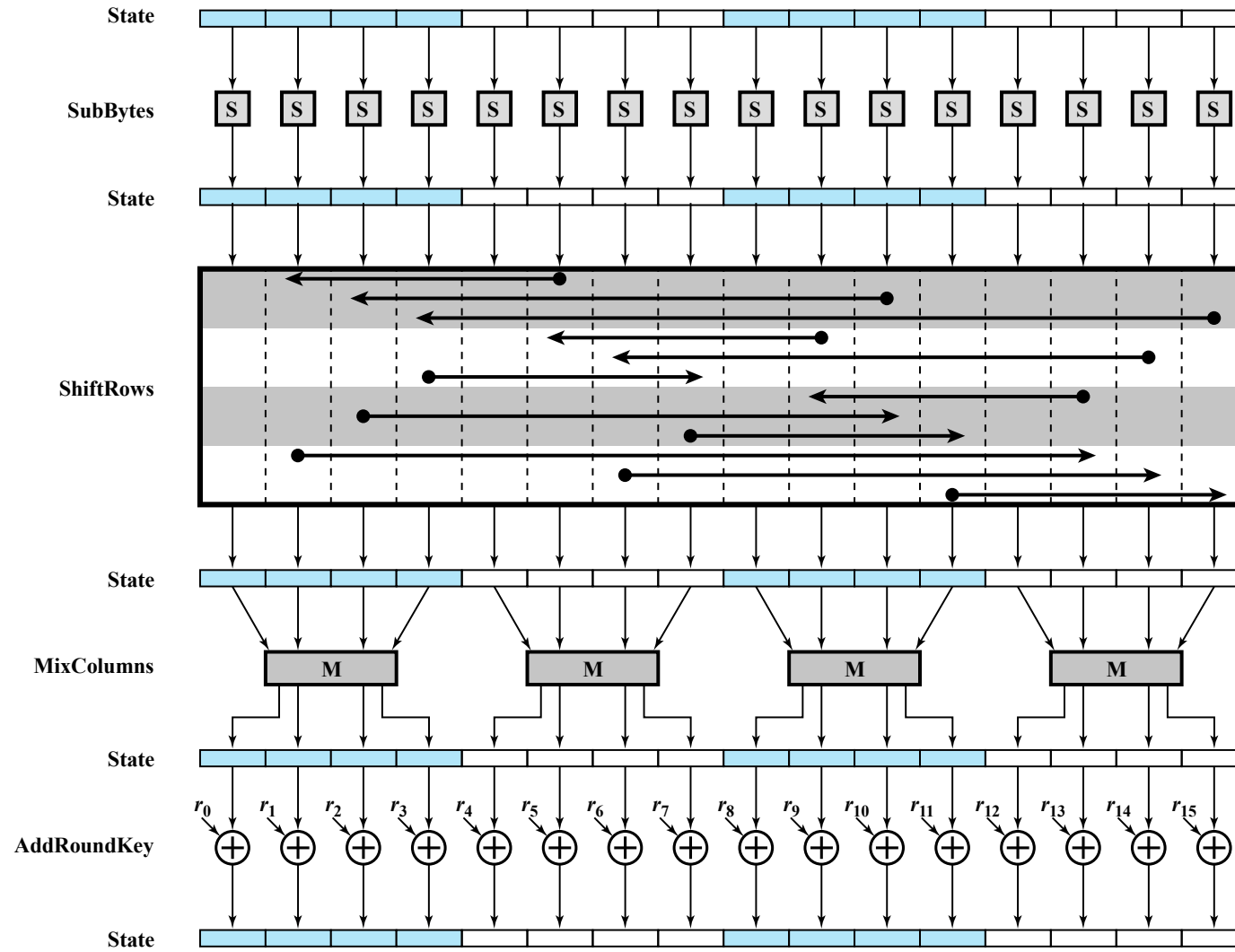


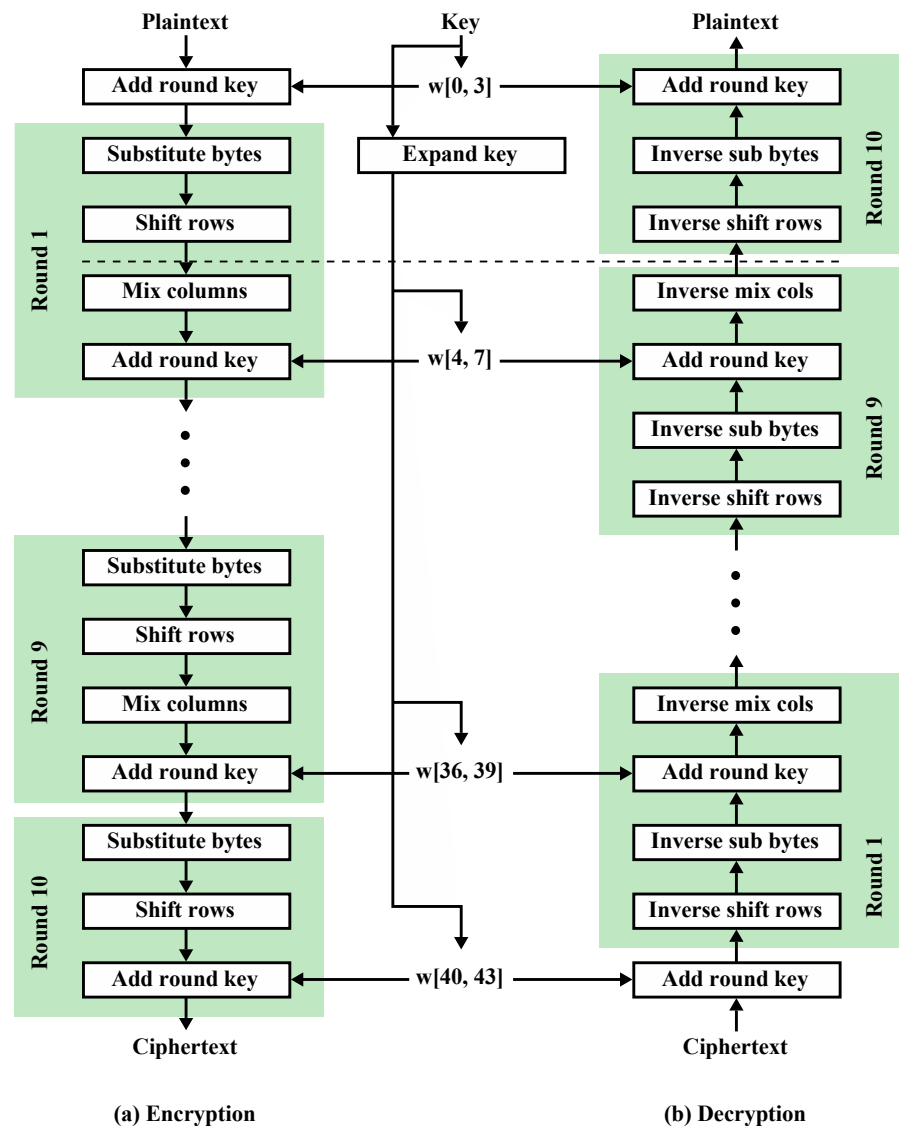
Advanced Encryption Standard (AES)

Input block size: 128 bits

Key length: 128 bits, 192bits, 256bits

AES Encryption Round





Key expand function:

Input: a 16-byte (128 bits) key

Output: a 176-byte key (44 words for 10 rounds + preliminary round)

Add round key function

Input: a 16-byte data array and a 16-byte key

AES has 10 rounds for 128-bit keys, **12 rounds** for 192-bit keys, and **14 rounds** for 256-bit keys.

$W[0,3]$

Original 4-word key for prelim. round

$W[4,7]$

Expanded key for round 1

$W[40,43]$

Expanded key for round 10

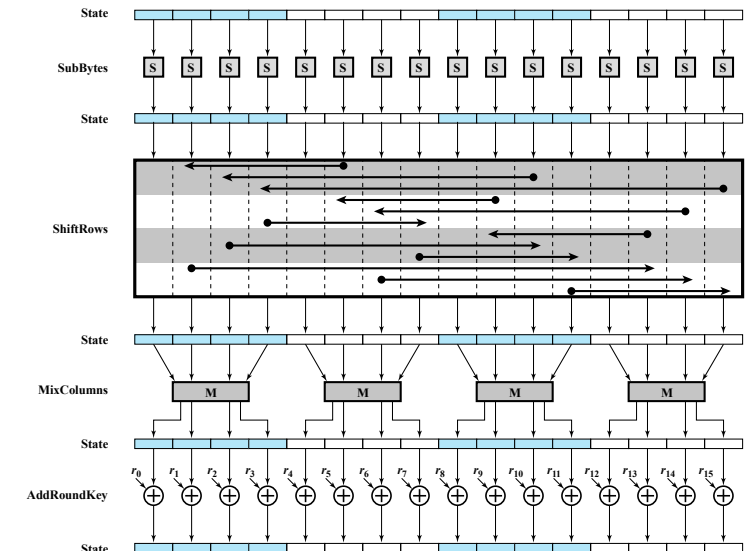
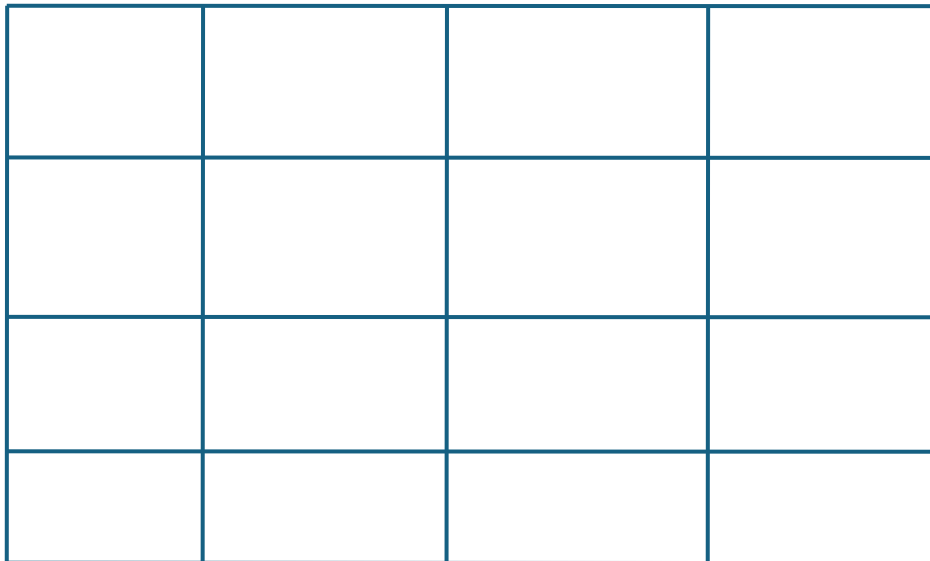
Figure 20.3 AES Encryption and Decryption with a key of 128 bits

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>

AES Algorithm details

- Imagine you are implementing AES encryption. The following 128-bit (16-byte) block is the state after the initial AddRoundKey step (i.e., plaintext $\oplus K_0$), ready to enter Round 1:

1A 93 57 9A 63 68 95 F1 1C 1A 78 6E 99 99 B1 1A



Substitute bytes

- Encryption uses S-Box



Table 20.2 AES S-Boxes

(a) S-box

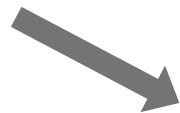
		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

E.g., DE => 1D

Substitute bytes

- Encryption uses S-Box

1A	63	1C	99
93	68	1A	99
57	95	78	B1
9A	F1	6E	1A



A2			
DC			

Table 20.2 AES S-Boxes

(a) S-box

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Shift Rows

- Encryption
 - For the first row of state, it is not altered.
 - For the second row, a 1-byte circular **left** shift is performed.
 - For the third row, a 2-byte circular **left** shift is performed.
 - For the fourth row, a 3-byte circular **left** shift is performed.
- Decryption
 - Similar. But use 1/2/3 circular right shift.

Encryption:

A2	FB	9C	EE
DC	45	A2	EE
5B	2A	BC	C8
B8	A1	9F	A2



A2	FB	9C	EE
45	A2	EE	DC

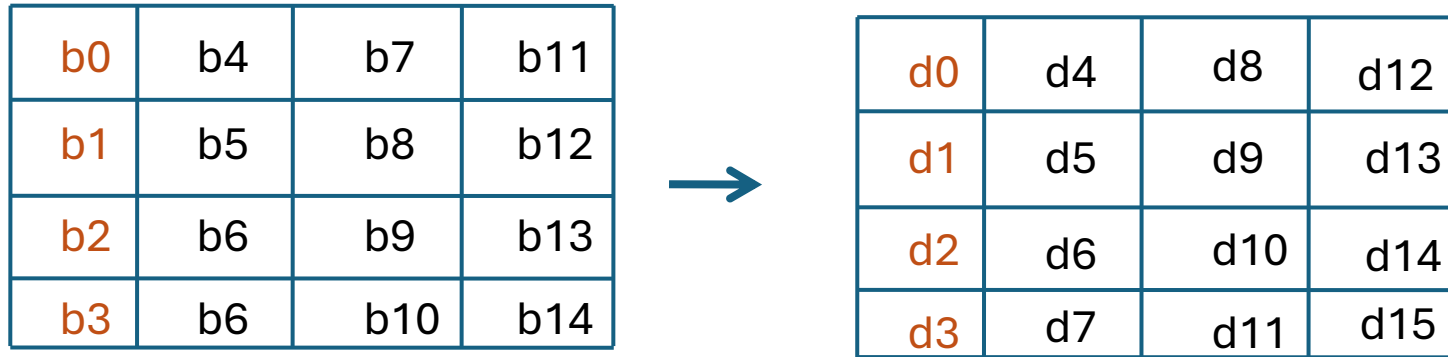
Mix Columns

- Operates on each column individually
- Mapping each byte to a new value that is a function of all four bytes in the column
- Use of equations over finite fields
- To provide good mixing of bytes in column



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

Mix Columns -Encryption



$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} \Rightarrow \begin{aligned} d_0 &= 2 \bullet b_0 \oplus 3 \bullet b_1 \oplus 1 \bullet b_2 \oplus 1 \bullet b_3 \\ d_1 &= 1 \bullet b_0 \oplus 2 \bullet b_1 \oplus 3 \bullet b_2 \oplus 1 \bullet b_3 \\ d_2 &= 1 \bullet b_0 \oplus 1 \bullet b_1 \oplus 2 \bullet b_2 \oplus 3 \bullet b_3 \\ d_3 &= 3 \bullet b_0 \oplus 1 \bullet b_1 \oplus 1 \bullet b_2 \oplus 2 \bullet b_3 \end{aligned}$$

*Multiplied by 1 => do nothing

*Multiplied by 2 => shift to left by 1 bit, padding is 0

* Multiplied by 3 => (Multiply*2) $\oplus$ (Multiply*1)

*Note that if the result of multiplication is larger than 11111111 (255),

You need remove the leftmost bit and XORed with 00011011

=> output = remove_leftmost(result) $\oplus$ 00011011

Operations in the Galois
finite field GF(2⁸)

A	B	A $\oplus$ B
0	0	0
0	1	1
1	0	1
1	1	0

$\oplus$ Exclusive OR

● Multiplication in GF (2⁸) and
irreducible polynomial is
 $X^8+X^4+X^3+X+1$

$10001001 = X^7 + X^3 + 1$

$01011001 = X^6 + X^4 + X^3 + 1$

Mix Columns –Encryption (Example)

A2	FB	9C	EE
45	A2	EE	DC
BC	C8	5B	2A
A2	B8	A1	9F

d0	d4	d8	d12
d1	d5	d9	d13
d2	d6	d10	d14
d3	d7	d11	d15

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$\begin{aligned} d_0 &= 2 \bullet b_0 \oplus 3 \bullet b_1 \oplus 1 \bullet b_2 \oplus 1 \bullet b_3 \\ d_1 &= 1 \bullet b_0 \oplus 2 \bullet b_1 \oplus 3 \bullet b_2 \oplus 1 \bullet b_3 \\ d_2 &= 1 \bullet b_0 \oplus 1 \bullet b_1 \oplus 2 \bullet b_2 \oplus 3 \bullet b_3 \\ d_3 &= 3 \bullet b_0 \oplus 1 \bullet b_1 \oplus 1 \bullet b_2 \oplus 2 \bullet b_3 \end{aligned}$$

$$\begin{aligned} \mathbf{d0} &= 2 \cdot \mathbf{A2} \oplus 3 \cdot \mathbf{45} \oplus 1 \cdot \mathbf{BC} \oplus 1 \cdot \mathbf{A2} = \mathbf{5F} \oplus \mathbf{CF} \oplus \mathbf{BC} \oplus \mathbf{A2} = \mathbf{8E} \\ \mathbf{d1} &= 1 \cdot \mathbf{A2} \oplus 2 \cdot \mathbf{45} \oplus 3 \cdot \mathbf{BC} \oplus 1 \cdot \mathbf{A2} = \mathbf{A2} \oplus \mathbf{8A} \oplus \mathbf{DF} \oplus \mathbf{A2} = \mathbf{55} \\ \mathbf{d2} &= 1 \cdot \mathbf{A2} \oplus 1 \cdot \mathbf{45} \oplus 2 \cdot \mathbf{BC} \oplus 3 \cdot \mathbf{A2} = \mathbf{A2} \oplus \mathbf{45} \oplus \mathbf{63} \oplus \mathbf{FD} = \mathbf{79} \\ \mathbf{d3} &= 3 \cdot \mathbf{A2} \oplus 1 \cdot \mathbf{45} \oplus 1 \cdot \mathbf{BC} \oplus 2 \cdot \mathbf{A2} = \mathbf{FD} \oplus \mathbf{45} \oplus \mathbf{BC} \oplus \mathbf{5F} = \mathbf{5B} \end{aligned}$$

CMSC 687/487: Introduction to Network Security

25

Mix Columns Example – Calculating d0

- $d0 = 2 \cdot A2 \oplus 3 \cdot 45 \oplus 1 \cdot BC \oplus 1 \cdot A2 = 5F \oplus CF \oplus BC \oplus A2 = 8E$

$2 * A2 = ?$

- **Step 1.** Get the binary form of A2, $A2 = 1010\ 0010$
- **Step 2.** Shift left by 1 bit (drop the leftmost bit, append a 0 on the right)
 $1010\ 0010 \rightarrow \mathbf{0100\ 0100} = 44$
The bit that was dropped is the leftmost bit, which was **1**.
- **Step 3.** Because the dropped bit was 1, XOR with 1B ($1B = 0001\ 1011$)

So, $2 * A2 = 5F$

$$\begin{array}{rcl} 0100\ 0100 & (44) \\ \oplus\ 0001\ 1011 & (1B) \\ \hline 0101\ 1111 & =\ 5F \end{array}$$

Mix Columns Example – Calculating d0

* Multiplied by 3 => (Multiply*2) (Multiply*1)

- $d0 = 2 \cdot A2 \oplus 3 \cdot 45 \oplus 1 \cdot BC \oplus 1 \cdot A2 = 5F \oplus CF \oplus BC \oplus A2 = 8E$

- $3 \cdot 45 = ?$

- **Step 1.** Rewrite $3 \cdot 45$ as $2 \cdot 45 \oplus 1 \cdot 45$, because $3 = 2 \oplus 1$ in $GF(2^8)$

So we first compute $2 \cdot 45$, then XOR the result with 45 itself

- **Step 2.** Get the binary form of 45, $45 = 0100\ 0101$

- **Step 3.** Shift left by 1 bit (drop the leftmost bit, append a 0 on the right):

$$0100\ 0101 \rightarrow 1000\ 1010 = 8A$$

The dropped leftmost bit was 0

- **Step 4.** Because the dropped bit was 0, there is no XOR with 1B. So $2 \cdot 45 = 8A$

- **Step 5.** Now XOR $2 \cdot 45$ with 45 to get $3 \cdot 45$

So $3 \cdot 45 = CF$

$$\begin{array}{rcl} & 1000\ 1010 & (8A = 2 \cdot 45) \\ \oplus & 0100\ 0101 & (45) \\ \hline & 1100\ 1111 & = CF \end{array}$$

Mix Columns Example – Calculating d0

$$d0 = 2 \cdot A2 \oplus 3 \cdot 45 \oplus 1 \cdot BC \oplus 1 \cdot A2 = 5F \oplus CF \oplus BC \oplus A2 = 8E$$

- $1 \cdot BC = 1011 \ 1100$

- $1 \cdot A2 = 1010 \ 0010$

- $d0 =$

0101	1111	(5F = 2 · A2)
⊕ 1100	1111	(CF = 3 · 45)
⊕ 1011	1100	(BC = 1 · BC)
⊕ 1010	0010	(A2 = 1 · A2)

1000	1110	= 8E

AddRoundKey

$$\text{state}'[r,c] = \text{state}[r,c] \oplus \text{roundkey}[r,c]$$

- Add round key – The round key is a 16-byte key – make it a 4x4 array first
 - Simply XOR State with bits of expanded key
 - Security from complexity of round key expansion and other stages of AES

0100	0111	(47)
⊕ 1010	1100	(AC)

1110	1011	= EB

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

 $\oplus$

AC	19	28	57
77	FA	D1	5C
66	DC	29	00
ED	A5	A6	BC

 $=$

EB	59	8B	1B
40	2E	A1	C3
F2	38	13	42
1E	84	E7	D2

e.g., the result after MixColumns

e.g., the round key

AES -Encryption

Input: 16-byte plaintext block, expanded round keys RK0 ... RK10

Initial round:

AddRoundKey(RK0)

Rounds 1 – 9 (each round):

- SubBytes – replace each byte using the S-box
- ShiftRows – row r is rotated LEFT by r bytes ($r = 0,1,2,3$)
- MixColumns – multiply each column by $\begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix}$ in $GF(2^8)$
- AddRoundKey(RK i) – XOR the state with round key i

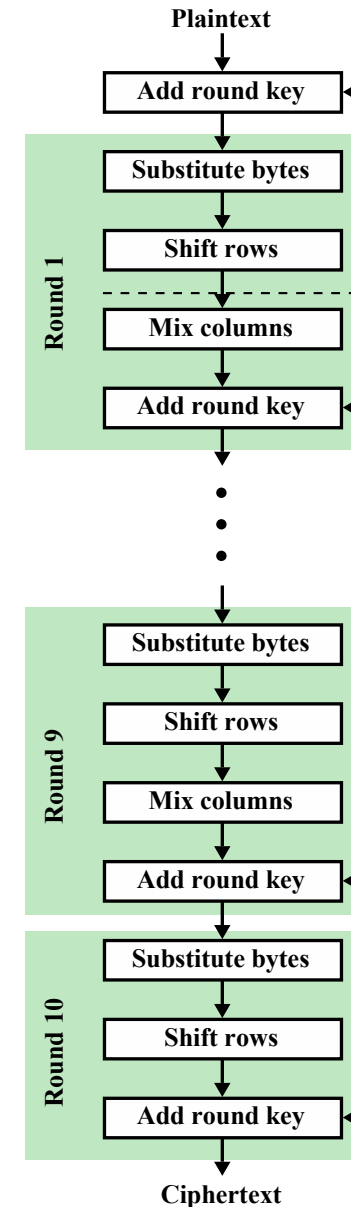
Final round (round 10):

SubBytes

ShiftRows

AddRoundKey(RK10) ← no MixColumns in the last round

Output: 16-byte ciphertext block



AES -Decryption

Input: 16-byte ciphertext block, expanded round keys RK0 ... RK10

Initial round:

AddRoundKey(RK10)

Rounds 9 – 1 (each round):

- InvShiftRows – row r is rotated RIGHT by r bytes ($r = 0,1,2,3$)
- InvSubBytes – replace each byte using the inverse S-box
- AddRoundKey(RK $_i$) – XOR the state with round key i
- InvMixColumns – multiply each column by [0E 0B 0D 09; 09 0E 0B 0D; 0D 09 0E 0B; 0B 0D 09 0E] in $GF(2^8)$

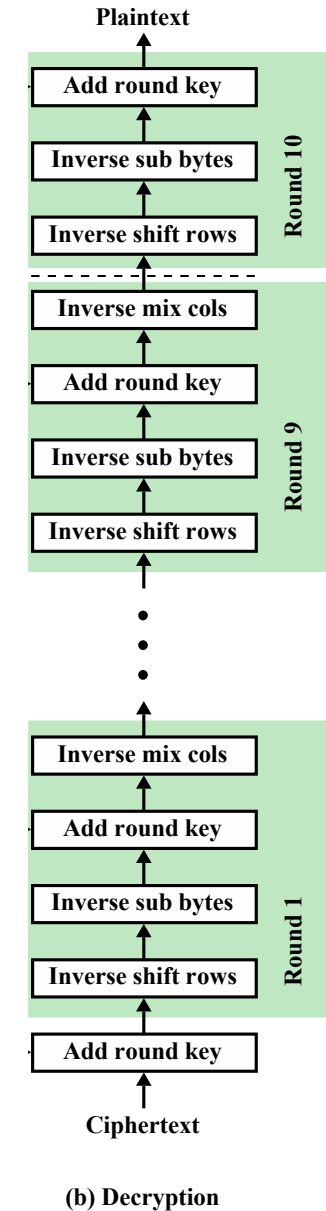
Final round:

InvShiftRows

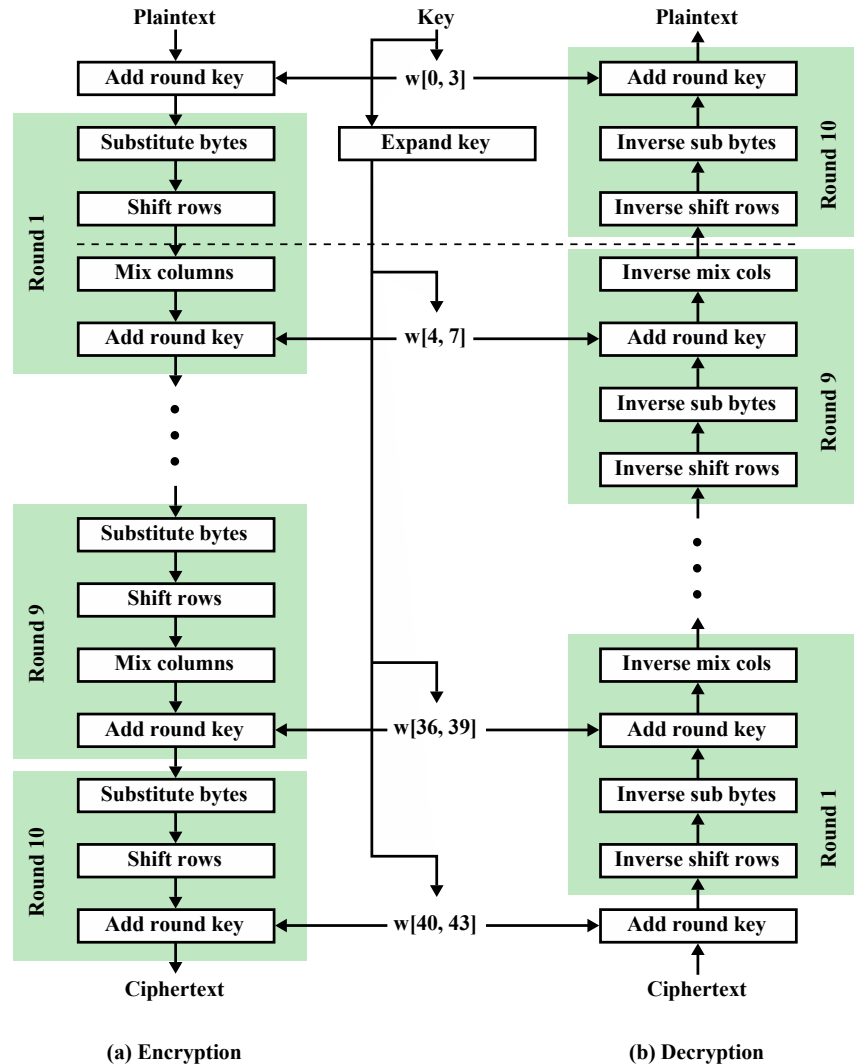
InvSubBytes

AddRoundKey(RK0) ← no InvMixColumns in the last round

Output: 16-byte plaintext block



AES



Side-by-side comparison

	Encryption	Decryption
Number of rounds	10	10
Round key order	RK0 → RK10	RK10 → RK0
Byte substitution	SubBytes (S-box)	InvSubBytes (inverse S-box)
Row operation	ShiftRows - rotate left by 0, 1, 2, 3	InvShiftRows - rotate right by 0, 1, 2, 3
Column operation	MixColumns - coefficients 02, 03, 01, 01	InvMixColumns - coefficients 0E, 0B, 0D, 09
Key mixing	AddRoundKey (XOR)	AddRoundKey (XOR) - identical, since XOR is its own inverse
Order within a round	SubBytes → ShiftRows → MixColumns → AddRoundKey	InvShiftRows → InvSubBytes → AddRoundKey → InvMixColumns
Last round	Omits MixColumns	Omits InvMixColumns

Substitute bytes -Decryption

- Decryption uses inverse S-Box

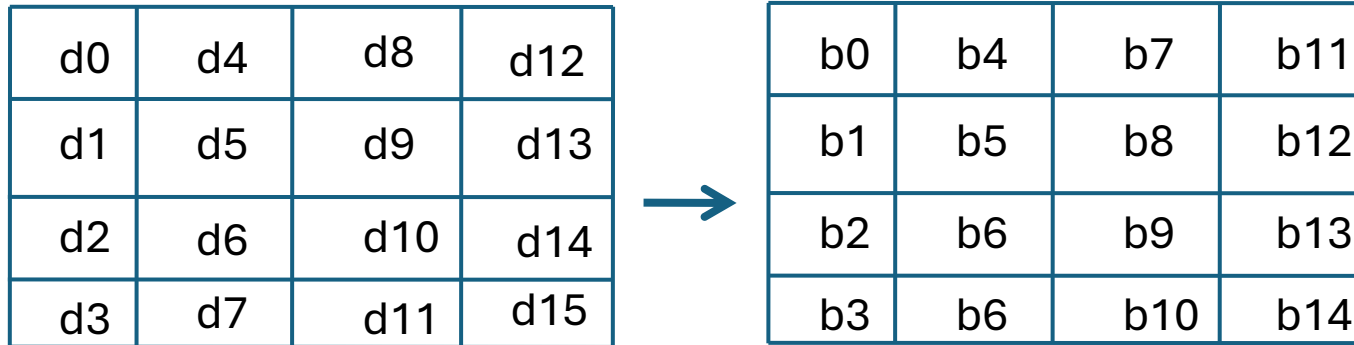
(b) Inverse S-box



		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

E.g., 1D => DE

Mix Columns -Decryption



$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix}$$

$$b_0 = 0E \cdot d_0 \oplus 0B \cdot d_1 \oplus 0D \cdot d_2 \oplus 09 \cdot d_3$$

$$b_1 = 09 \cdot d_0 \oplus 0E \cdot d_1 \oplus 0B \cdot d_2 \oplus 0D \cdot d_3$$

$$b_2 = 0D \cdot d_0 \oplus 09 \cdot d_1 \oplus 0E \cdot d_2 \oplus 0B \cdot d_3$$

$$b_3 = 0B \cdot d_0 \oplus 0D \cdot d_1 \oplus 09 \cdot d_2 \oplus 0E \cdot d_3$$



Block Cipher Modes of Operation

Block Cipher Modes of Operation

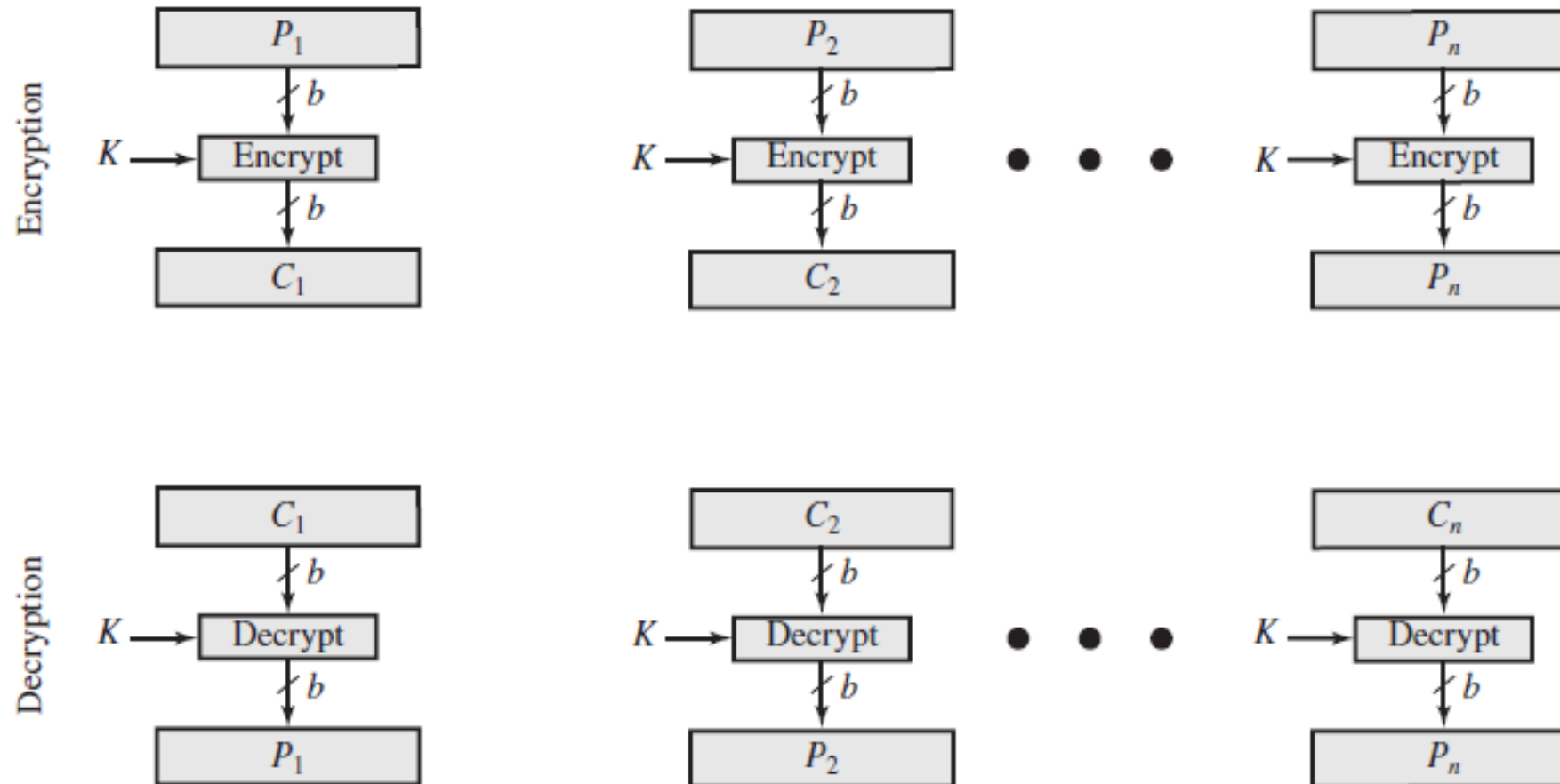
Mode	Description	Typical Application
Electronic Codebook (ECB)	Each block of 64 plaintext bits is encoded independently using the same key.	•Secure transmission of single values (e.g., an encryption key)
Cipher Block Chaining (CBC)	The input to the encryption algorithm is the XOR of the next 64 bits of plaintext and the preceding 64 bits of ciphertext.	•General-purpose block-oriented transmission •Authentication
Cipher Feedback (CFB)	Input is processed s bits at a time. Preceding ciphertext is used as input to the encryption algorithm to produce pseudorandom output, which is XORed with plaintext to produce next unit of ciphertext.	•General-purpose stream-oriented transmission •Authentication
Output Feedback (OFB)	Similar to CFB, except that the input to the encryption algorithm is the preceding DES output.	•Stream-oriented transmission over noisy channel (e.g., satellite communication)
Counter (CTR)	Each block of plaintext is XORed with an encrypted counter. The counter is incremented for each subsequent block.	•General-purpose block-oriented transmission •Useful for high-speed requirements

(64bits for DES, 128 bits for AES)

Electronic Codebook (ECB)

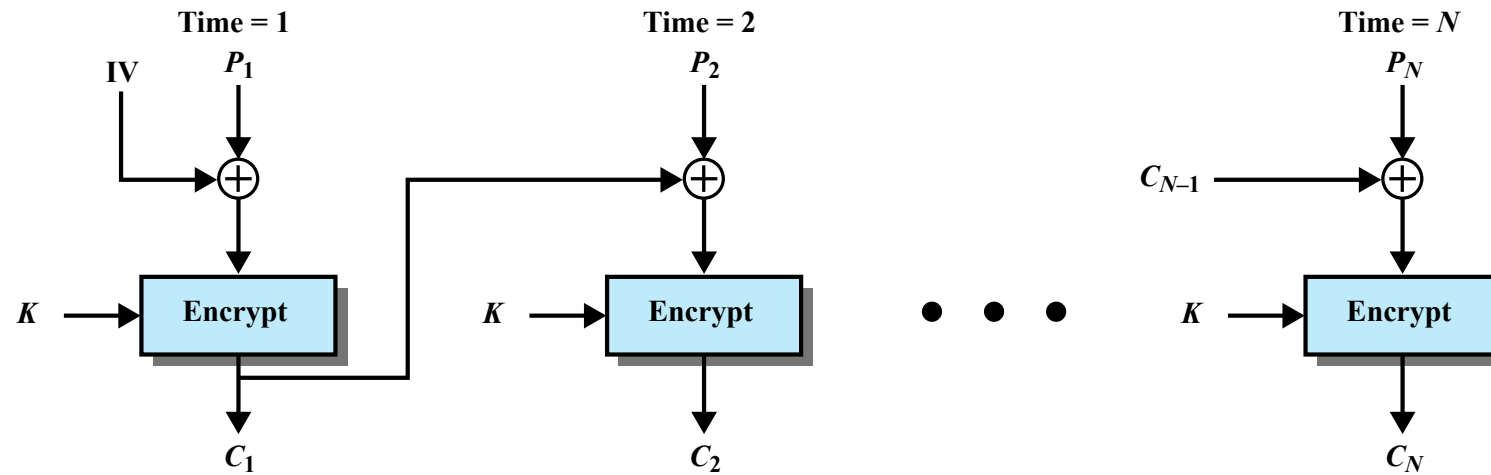
- Simplest mode
- Plaintext is handled b bits at a time and **each block is encrypted using the same key**
- “Codebook” because have a unique ciphertext value for each plaintext block
 - Not secure for long messages since repeated plaintext is seen in repeated ciphertext
- To overcome security deficiencies you need a technique where the same plaintext block, if repeated, produces different ciphertext blocks

Electronic Codebook (ECB)

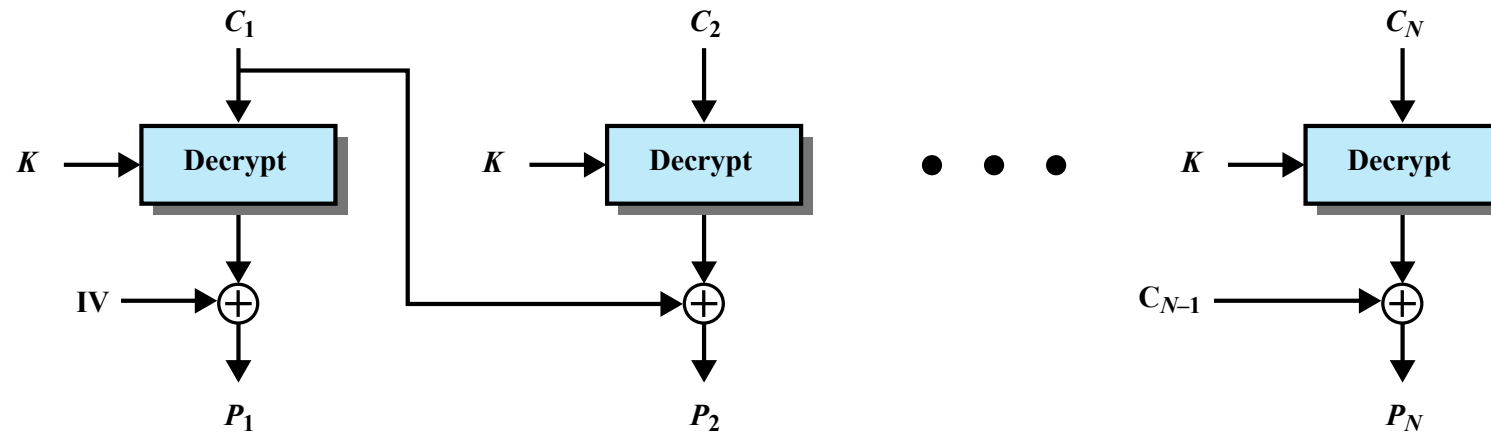


(a) Block cipher encryption (electronic codebook mode)

Cipher Black Chaining (CBC)



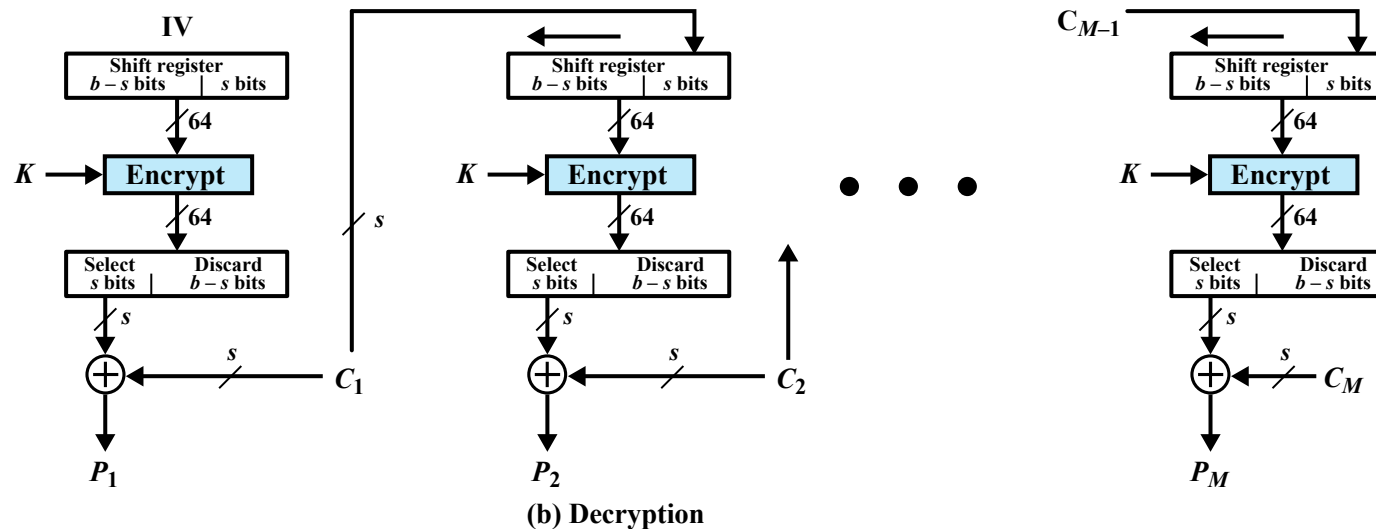
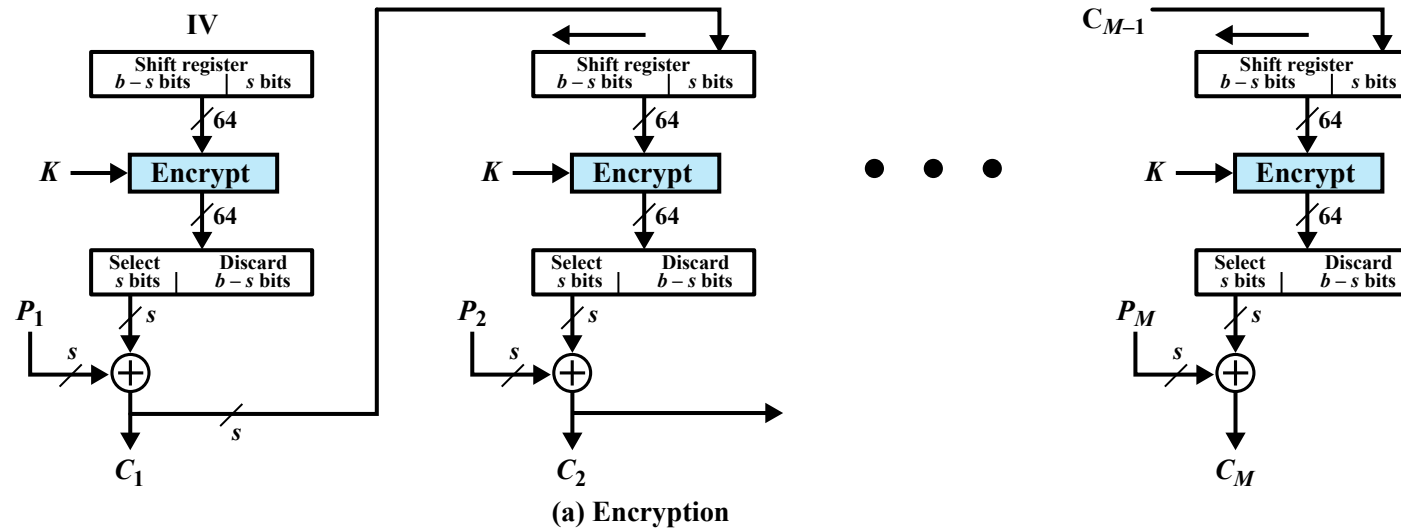
(a) Encryption



(b) Decryption

IV: Initial vector,
which is only
known to sender
and receiver.

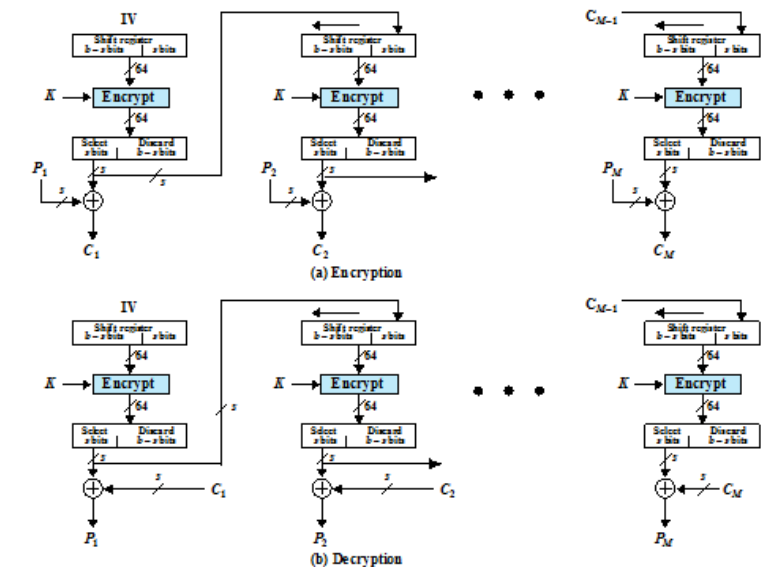
s-bit Cipher Feedback (CFB)



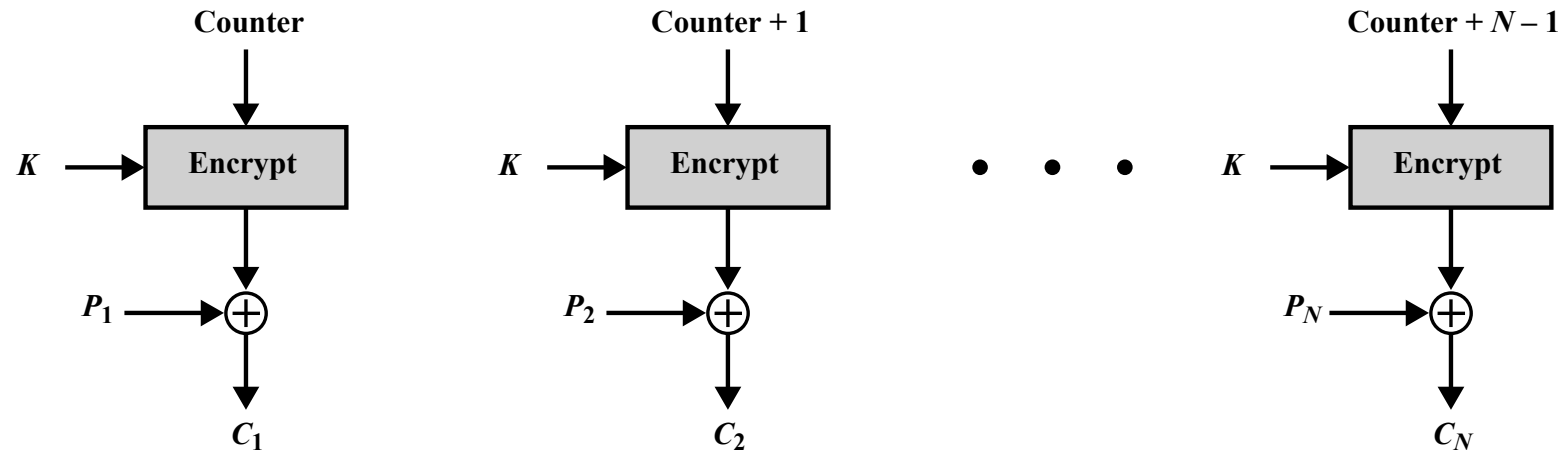
All block cipher algorithms can be converted to stream cipher based on CFB

Output Feedback

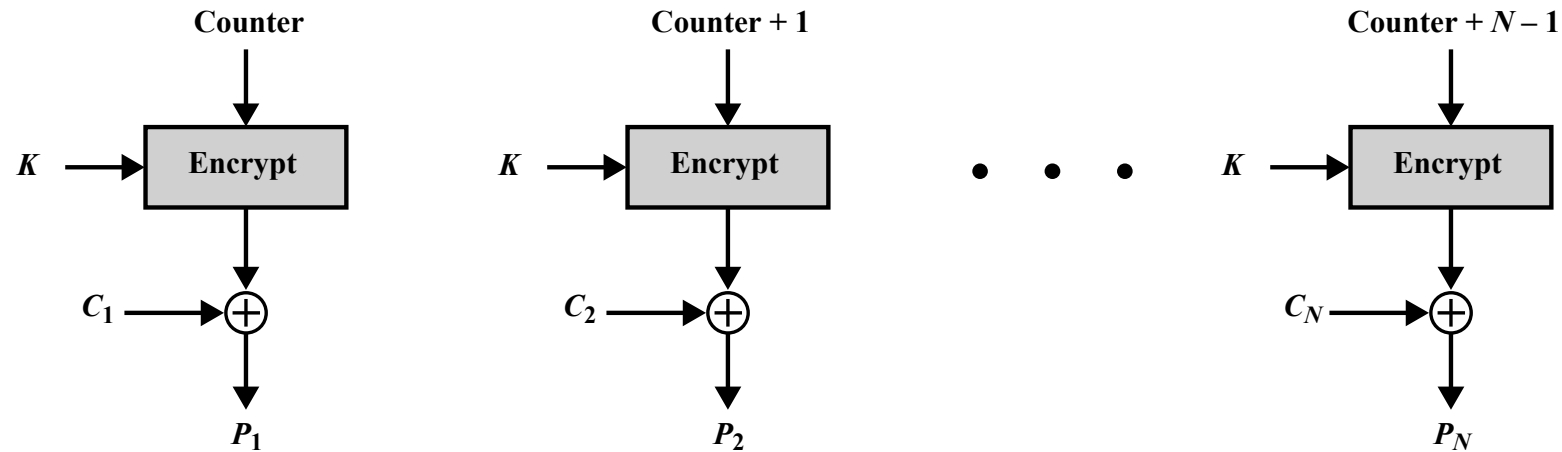
- Similar to cipher feedback, but the input to the shift register
 - Output feedback uses the **proceeding s-bit key** but Cipher feedback uses the **proceeding s-bit ciphertext**
 - Using Output Feedback mode, encryption and decryption of i th s-bit data do not rely on $i-1$ th s-bit data (i.e., **no data chaining**)



Counter (CTR) Mode



(a) Encryption



(b) Decryption